

1. Aufgabe *Designers quest for the best: Clock Tree*

Chip-Entwürfe enthalten eine immer größer werdende Zahl von Gattern und Flipflops. Für dieses Beispiel gehen wir von einem Design mit 10000 Flipflops aus. Die Flipflops werden *alle* vom selben Taktsignal (`clk`) gesteuert. Es handelt sich also um einen synchronen Entwurf, wie man ihn auch in der Praxis antrifft.

Das Taktsignal wird von einem Taktgenerator erzeugt. Wenn man aber den Ausgang dieses Taktgenerators einfach am Eingang der 10000 Flipflops anschliessen würde, so wäre der Taktgenerator hoffnungslos überlastet. Es stünde in diesem Fall kein brauchbares Taktsignal mehr zur Verfügung.

Abhilfe schafft ein *Clock Tree*. Der Taktgenerator speist die Flipflops hierbei nicht unmittelbar mit dem Taktsignal. Stattdessen wird eine Anzahl Flipflops von einem Pufferverstärker (*buffer*) aus mit dem Takt versorgt. Dies könnten z.B. 10 Flipflops sein. Damit würden also 1000 solcher *buffer* benötigt. Auch diese Last von 1000 Eingängen ist für den Taktgenerator zu viel. Also versorgt man einfach je 10 der *buffer* wiederum von einem weiteren *buffer* aus. Da auch die somit notwendigen 100 *buffer* eine zu große Last für den Taktgenerator darstellen, wird noch einen weitere Stufe eingefügt. Da die entstehende Struktur das Bild eines Baums ergibt, spricht man auch von einem *clock tree*.

- (a) Der Takt durchläuft vom Taktgenerator kommend eine Reihe der *buffer*. Modelliere einen Weg, den das Taktsignal vom Generator aus bis zu *einem* der Flipflops nimmt. Jeder der drei *buffer* wird mit einem eigenen *process* modelliert. Der Taktgenerator bekommt im Modell einen eigenen *process* und erzeugt das *signal clk* mit einer Taktfrequenz von 100 MHz. Ein Flipflop soll nicht modelliert werden.
- (b) Für jeden *buffer* ist eine Verzögerungszeit von 7 ns zu modellieren. Als Verzögerungsmodell eines einzelnen *buffer* ist *transport delay* zu verwenden. Welche Taktfrequenz wird von Ihrem Modell des *clock tree* maximal an die Flipflops weitergeleitet? Finde diese per Simulation und erkläre das Ergebnis.
- (c) Wie vorher, aber diesmal folgen die *buffer* dem Verzögerungsmodell *inertial delay* mit einem *rejection limit* von 7 ns. Wie hoch ist diesmal die maximale Taktfrequenz? Simuliere wieder und erläutere auch diesmal das Ergebnis.
- (d) Nun sollen alle *buffer* als Gesamtheit mit nur einem *process B1_3* modelliert werden. Modelliere dazu einen neuen *process*, der von der Taktleitung `clk` angesteuert wird, die Verzögerungszeiten der *buffer*-Kette nachbildet und ein *signal y4* erzeugt.

Simuliere die *buffer*-Kette und den *process B1_3* gemeinsam und vergleiche die Ausgaben im *Waveform*-Fenster. Führe Simulationen mit den beiden Verzögerungszeitmodellen von VHDL durch. Welches Modell ergibt das genauere Ergebnis gemessen an den beiden vorhergehenden Versuchen?

2. Aufgabe Resolved Signals

Abbildung 1 zeigt die Schalterdarstellung einer Open-Kollektor Transistorschaltung, die ein NOR-Gatter (Eingangsvariablen: a, b; Ausgangsvariable: c) realisiert. Die parallelen Schalter können dabei nur einen definierten '0'-Wert erzeugen; der Wert '1' kann nicht erzwungen (also ausgegeben) werden; statt '1' wird der Ausgang der Parallelschaltung hochohmig. Nur durch einen Pull-up Widerstand liegt in diesem Fall am Punkt c eine logische '1' an.

- (a) Modelliere diese Schaltung in VHDL mittels *Concurrent Signal Assignments* (ein *Assignment* pro Schalter und ein *Assignment* für den Pull-Up Widerstand) und unter Verwendung des Datentyps `std_logic` für den Schaltungsknoten an welchem die Schalter und der Pull-Up Widerstand angeschlossen sind.
- (b) Mittels einer entsprechenden *Testbench* soll die Funktion der Schaltung nachgewiesen werden.

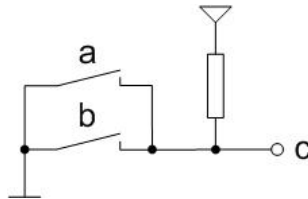


Abbildung 1: Resolved Signals